WARWICK
THE UNIVERSITY OF WARWICK

Lecture 8
Part-of-Speech Tagging – Hidden Markov Model

Gabriele Pergola

Natural Language Processing
CS918

February 2026

CAN WE HAVE A LANGUAGE WITHOUT VERBS?

OR WITHOUT NOUNS?

[1] Evans, N. 2000. "*Word classes in the world's languages*".
[2] Gil, D. 2000. "Syntactic categories, cross-linguistic variation and universal grammar".

KEEP
CALM
AND
CARRY
ON

KEEP
CALM
AND
ASK
QUESTIONS
KeepCalmAndPosters.com

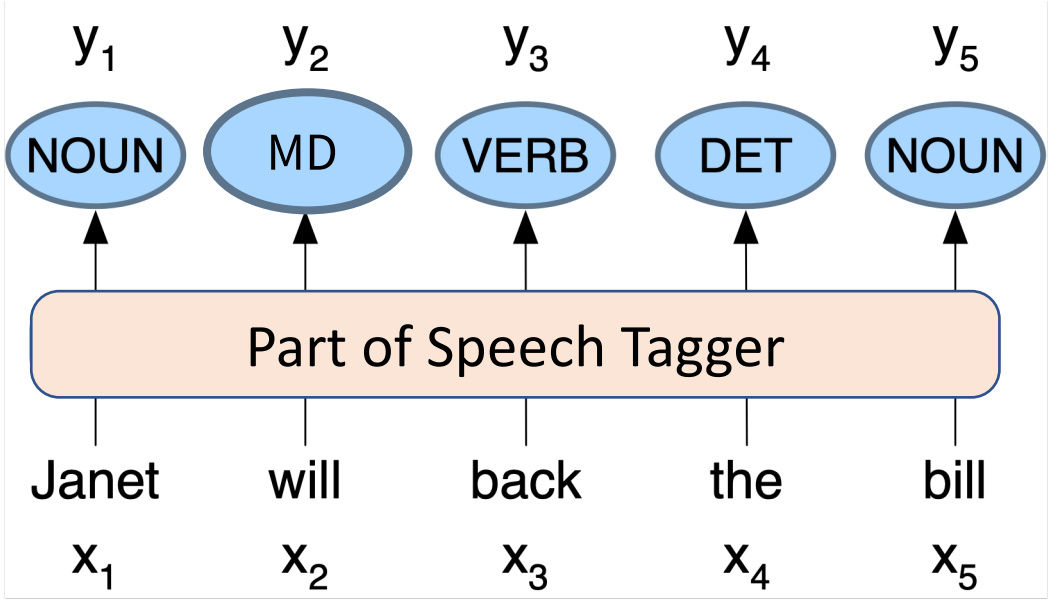
KEEP
CALM
AND
TRY
AGAIN
www.unitelvoice.com

Outline

- Part-of-Speech Tagging
  - Why is it hard?
- Sequence Classifiers
  - Hidden Markov Models (HMM)
    - Introduction to HMM
    - Markov Process
    - Learning of HMM

Part-of-Speech (POS) Tagging - Sequence Classification

- Given a sequence of words, we want to **classify the part-of-speech (i.e., grammatical roles**) of each words.
  - Part-of-Speech (POS) tagging
    Assigning **categories** to words, e.g. *noun, verb, or an adjective*



Word Classes: Tag Sets

- **Vary** in number of tags: a dozen to over 200
- Size of tag sets **depends on** language, objectives and purpose

WORD	LEMMA	TAG
the	the	+DET
girl	girl	+NOUN
kissed	kiss	+VPAST
the	the	+DET
boy	boy	+NOUN
on	on	+PREP
the	the	+DET
cheek	cheek	+NOUN

Word Classes: **The Penn Treebank Tag Set**

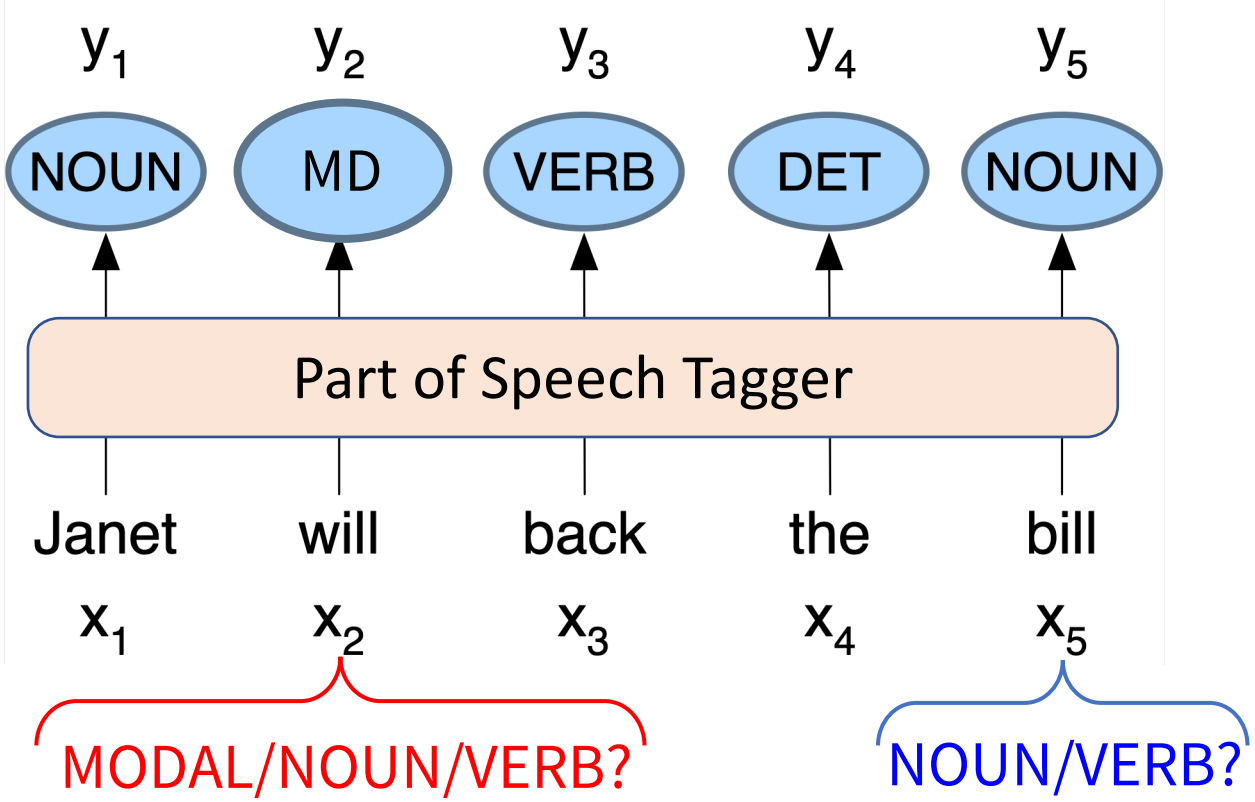
Tag	Description	Example	Tag	Description	Example	Tag	Description	Example
CC	coordinating conjunction	*and, but, or*	PDT	predeterminer	*all, both*	VBP	verb non-3sg present	*eat*
CD	cardinal number	*one, two*	POS	possessive ending	*’s*	VBZ	verb 3sg pres	*eats*
DT	determiner	*a, the*	PRP	personal pronoun	*I, you, he*	WDT	wh-determ.	*which, that*
EX	existential ‘there’	*there*	PRP$	possess. pronoun	*your, one’s*	WP	wh-pronoun	*what, who*
FW	foreign word	*mea culpa*	RB	adverb	*quickly*	WP$	wh-possess.	*whose*
IN	preposition/ subordin-conj	*of, in, by*	RBR	comparative adverb	*faster*	WRB	wh-adverb	*how, where*
JJ	adjective	*yellow*	RBS	superlatv. adverb	*fastest*	$	dollar sign	$
JJR	comparative adj	*bigger*	RP	particle	*up, off*	#	pound sign	#
JJS	superlative adj	*wildest*	SYM	symbol	+,%, &	“	left quote	‘ or “
LS	list item marker	*1, 2, One*	TO	“to”	*to*	”	right quote	’ or ”
MD	modal	*can, should*	UH	interjection	*ah, oops*	(	left paren	[, (, {, <
NN	sing or mass noun	*llama*	VB	verb base form	*eat*	)	right paren	],), }, >
NNS	noun, plural	*llamas*	VBD	verb past tense	*ate*	,	comma	,
NNP	proper noun, sing.	*IBM*	VBG	verb gerund	*eating*	.	sent-end punc	. ! ?
NNPS	proper noun, plu.	*Carolinas*	VBN	verb past part.	*eaten*	:	sent-mid punc	: ; ... – -

Why Part of Speech Tagging?

Can be useful for other NLP tasks:

- **Sentiment or affective tasks**: may want to distinguish adjectives or other Part-Of-Speech tags
- **Text-to-Speech** (how do we pronounce "object" or “lead”?)
- **MT**: reordering of adjectives and nouns (say from Spanish to English)
- **Parsing**: POS tagging can improve syntactic parsing

Why is POS hard?



- 'will' and 'bill' are **ambiguous**.
- We need to look at the context to classify the correct POS tagging for 'will' and 'bill'.
  - The **context** is given by the **sequence**

Why is POS hard?

- **Two probabilities** to determine POS of a word:

[1] P('will$_{MD}$') vs P('will$_{NN}$') → MOST-FREQUENT-TAG Baseline

[2] P('will$_{MD}$' | NN) vs P('will$_{NN}$' | NN) → BIGRAM TAGGING (i.e., Markov Chain)

Janet will back the bill

MODAL/NOUN/VERB? NOUN/VERB?

1st Tagging Strategy: **Most-Frequent-Tag Baseline**

- One simple strategy: **Most-Frequent-Tag Baseline**
  - Just assign the most common part of speech to **each word independently**.

E.g.: '*will*' is always tagged as a *modal* — never as a noun or verb.

$P(\text{'will}_{\mathbf{MD}}\text{'}) \gg P(\text{'will}_{NN}\text{'})$

Janet will back the bill

MODAL/NOUN/VERB **?**

1st Tagging Strategy: **Most-Frequent-Tag Baseline**

- One simple strategy: **Most-Frequent-Tag Baseline**
  - Just assign the most common part of speech to **each word independently**.

    E.g.: '*will*' is always tagged as a *modal* — never as a noun or verb.

    $P(\text{'will}_{\mathbf{MD}}\text{'}) \gg P(\text{'will}_{NN}\text{'})$

    Janet will back the bill

    MODAL

- Also called **unigram tagging**, since we only consider one token at a time.
- Surprisingly, even this crude approach typically gives around **90% accuracy**.

  (State-of-the-art is 96–98%).
- **Can we do better?**

Part-of-Speech (POS) Tagging – **Can we do better?**

- Ok, can we do better than the crude baseline?
  - Hidden Markov Models (HMM)
    - Bigram tagging (i.e., Markov Chain)
  - Maximum Entropy Markov Models (MEMM)
  - Conditional Random Fields (CRF)
  - Deep Learning
    - Recurrent Neural Networks (RNN)
    - Long Short-Term Memory Networks (LSTM)
    - Convolutional Neural Networks (CNN)
    - Transformers

Bigram Tagging
(i.e. Markov Chain)

Bigram Tagging (i.e. Markov Chain)

- **Two probabilities** to determine POS of a word:

[1] P('$will_{MD}$') vs P('$will_{NN}$') → MOST-FREQUENT-TAG Baseline

[2] P('MD' | NN) vs P('NN' | NN) → BIGRAM TAGGING (i.e., Markov Chain)

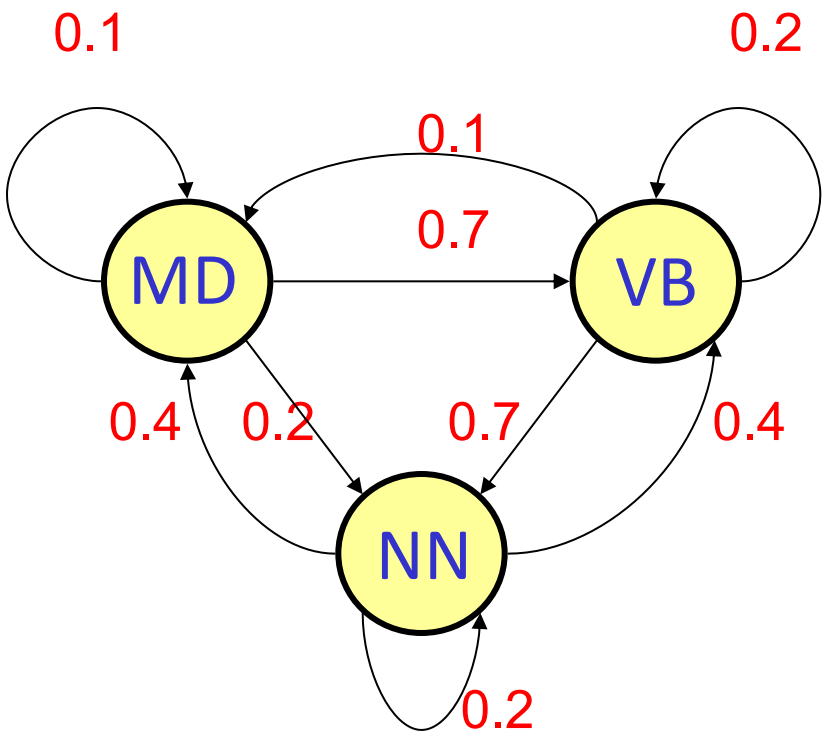
Janet will back the bill

MODAL/NOUN/VERB? NOUN/VERB?

Two ways to represent the conditional probability table of a first-order Markov process

Current State

Next State	MD	NN	VB
MD	0.1	0.4	0.1
NN	0.2	0.2	0.7
VB	0.7	0.4	0.2



Example sequence: NN MD VB NN

Markov chain = "First-order observable Markov Model"

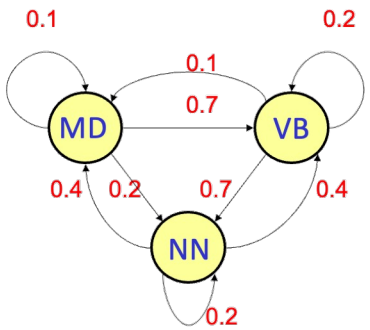
- A set of states
  - $Q = s_1, s_2 \ldots s_N$; the state at time t is s_t #(s_t is the part of text)

- Transition probabilities
  - a set of probabilities $A = a_{01} a_{02} \ldots a_{n1} \ldots a_{nn}$
  - Each a_{ij} represents the probability of transitioning from state i to state j
  - The set of these is the transition probability matrix A

- Markov Assumption
  - The probability of a tag only depends on the immediately preceding tag → **Markov Chain**

Current State

Next State	MD	NN	VB
MD	0.1	0.4	0.1
NN	0.2	0.2	0.7
VB	0.7	0.4	0.2



What does “Markov” mean

The next term in a sequence could **depend on all the previous terms**.

- But things are much **simpler** if it doesn’t!
- If it only depends on the previous term it is called “first-order” Markov.
- If it depends on the two previous terms it is second-order Markov.

- A **first order Markov process** for discrete symbols is defined by:
  - An initial probability distribution over symbols;

    &
  - A transition matrix composed of conditional probabilities

The probability of generating a string

Product of probabilities,
one for each term in the sequence

$$p(\{s_t\}_1^T) = p(s_1)\prod_{t=2}^{T} p(s_t \mid s_{t-1})$$

This means a **sequence of symbols** from time 1 to time T

This comes from the table of **initial probabilities**

This is a **transition** probability

Probability of a sequence - Example

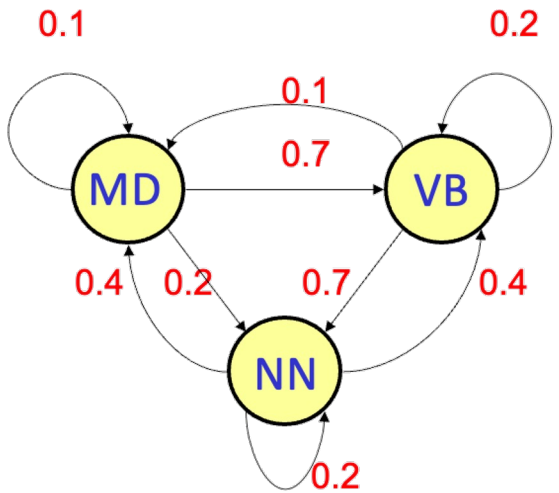
- What is the probability of the sequence **NN MD VB NN**?
  (e.g., *John will help Carl*)

- Suppose the probability of starting π is the same for all states (0.35)

$$P(\text{NN-MD-VB-NN}) = \pi_3\, a_{01}\, a_{20}\, a_{12} = 0.35 \times 0.4 \times 0.7 \times 0.7 = 0.06$$

Current State

Next State	MD	NN	VB
MD	0.1	0.4	0.1
NN	0.2	0.2	0.7
VB	0.7	0.4	0.2



Probability of a sequence - Example

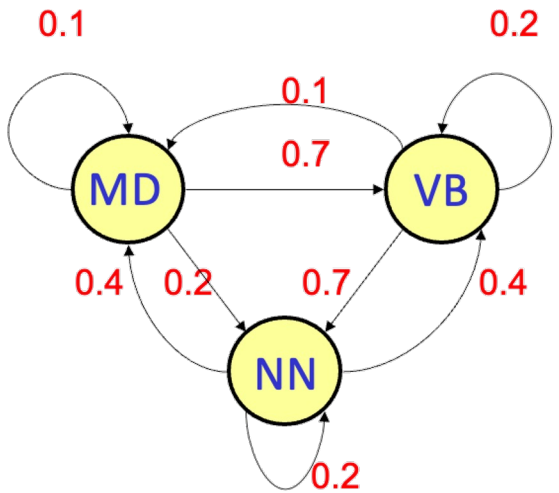
- What is the probability of the sequence **NN MD VB NN**?
  (e.g., ***John*** *will help Carl*)

- Suppose the **probability of starting** π is the same for all states (**0.35**)

$$P(\text{NN-MD-VB-NN}) = \pi_3\, a_{01}\, a_{20}\, a_{12} = 0.35 \times 0.4 \times 0.7 \times 0.7 = 0.06$$

Current State

Next State	MD	NN	VB
MD	0.1	0.4	0.1
NN	0.2	0.2	0.7
VB	0.7	0.4	0.2



Probability of a sequence - Example

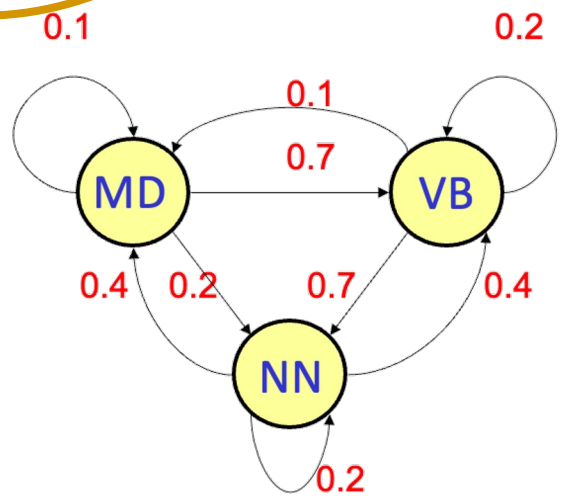
- What is the probability of the sequence **NN MD VB NN**?
  (e.g., *John* ***will*** *help Carl*)

- Suppose the probability of starting π is the same for all states (0.35)

$$P(\text{NN-MD-VB-NN}) = \pi_3\, a_{01}\, a_{20}\, a_{12} = 0.35 \times 0.4 \times 0.7 \times 0.7 = 0.06$$

Current State

Next State	MD	NN	VB
MD	0.1	0.4	0.1
NN	0.2	0.2	0.7
VB	0.7	0.4	0.2



Probability of a sequence - Example

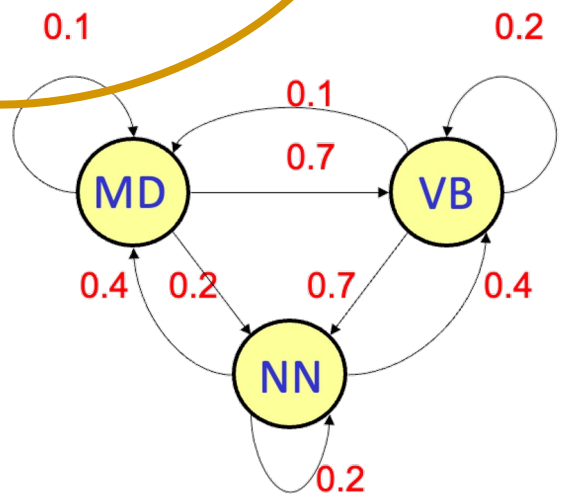
- What is the probability of the sequence **NN MD VB NN**?
  (e.g., *John will* ***help*** *Carl*)

- Suppose the probability of starting π is the same for all states (0.35)

$$P(\text{NN-MD-VB-NN}) = \pi_3\, a_{01}\, a_{20}\, a_{12} = 0.35 \times 0.4 \times 0.7 \times 0.7 = 0.06$$

Current State

Next State	MD	NN	VB
MD	0.1	0.4	0.1
NN	0.2	0.2	0.7
VB	0.7	0.4	0.2



Probability of a sequence - Example

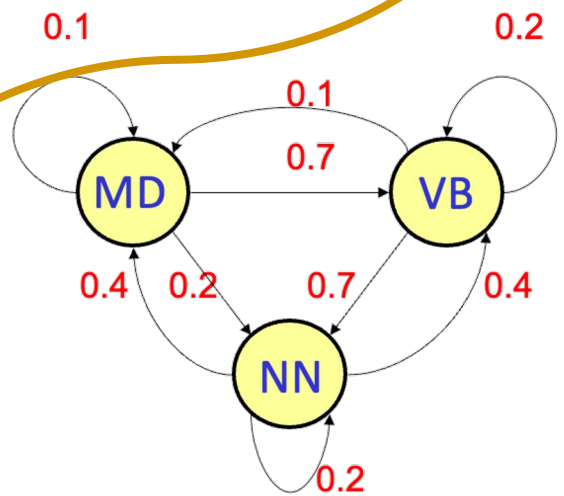
- What is the probability of the sequence **NN MD VB NN**?
  (e.g., *John will help* ***Carl***)

- Suppose the probability of starting π is the same for all states (0.35)

$$P(\text{NN-MD-VB-NN}) = \pi_3\, a_{01}\, a_{20}\, a_{12} = 0.35 \times 0.4 \times 0.7 \times 0.7 = 0.06$$

Current State

Next State	MD	NN	VB
MD	0.1	0.4	0.1
NN	0.2	0.2	0.7
VB	0.7	0.4	0.2



Recap - Tagging with Bigram (Markov Chain)

Recap

- When using bigram tagging for POS, you start with the **first word** and select its tag based on the **probability of that tag occurring at the beginning of a sentence** (initial state probability).

- For each subsequent word, you predict its tag based on the probability of that tag following the previous tag (this is the **transition probability** in the Markov Chain).

- These probabilities are often **estimated from a tagged corpus**, which is a large collection of text where each word has been tagged with its correct POS.

Learning the conditional probability table

Next State \ Current State	MD	NN	VB
MD			
NN		?	
VB			

- **Naïve**: Just observe a lot of strings and set the conditional probabilities equal to observed probabilities

$$p(B \mid A) = \frac{\sum_{strings} occurrences\ of\ \ AB}{\sum_{strings} occurrences\ of\ \ A}$$

- But do we really believe it if we get a **zero**?

- **Better**: **add 1** to top and number of symbols to bottom. This is like having a weak uniform prior over the transition probabilities.

$$p(B \mid A) = \frac{1 + \sum_{strings} \# AB}{N_{symbols} + \sum_{strings} \# A}$$

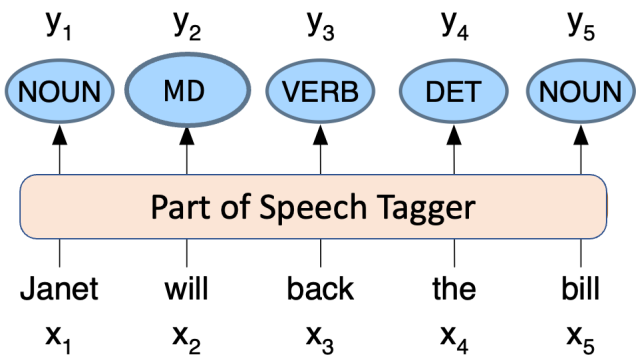
Problems with Bigram tagging (From Markov Chain to HMM)

- The probability of a word being assigned a particular tag can vary significantly depending on the **actual word**, **not just its preceding tag**.

P('Will'| MD) != P('Will' | NN)

- **Extension: Hidden Markov Models** consider both the **sequence of tags** and the **actual words** through **emission probabilities**.

- For Markov chains, the output symbols are the same as the states.
  - See MD: we're in state MD
- But in part-of-speech tagging (and other things)
  - The output symbols are **words**
  - But the hidden states are **part-of-speech tags**



Hidden Markov Model (HMM)

Discriminative and Generative Models

- We have training example $x^{(i)}, y^{(i)}$ for $i = 1 \ldots m$.
  Each $x^{(i)}$ is an **input**, each $y^{(i)}$ is a **label**.

- Task is to learn a function f mapping inputs x to labels $f(x)$

Discriminative models

- Learn a distribution $\boldsymbol{p}(\boldsymbol{y}|\boldsymbol{x})$ from training examples
- For any <u>test</u> input x, define $f(x) = \arg\max_{y} p(y|x)$

$\boldsymbol{p}(\boldsymbol{y}|\boldsymbol{x})$

Discriminator for my data

Discriminative and Generative Models

- We have training example $x^{(i)}, y^{(i)}$ for $i = 1 \ldots m$.
  Each $x^{(i)}$ is an **input**, each $y^{(i)}$ is a **label**.

- Task is to learn a function f mapping inputs x to labels $f(x)$

Generative models

- Learn a distribution $\boldsymbol{p}(\boldsymbol{x}, \boldsymbol{y})$ from training examples
- Often we have $\boldsymbol{p}(\boldsymbol{x}, \boldsymbol{y}) = p(y) * p(x|y)$
- We then have

$$p(y|x) = \frac{p(y)p(x|y)}{p(x)}$$

$\boldsymbol{p}(\boldsymbol{y}, \boldsymbol{x})$

Generator
for my data

Decoding with Generative Models

- We have training example $x^{(i)}, y^{(i)}$ for $i = 1 \ldots m$.
  Each $x^{(i)}$ is an **input**, each $y^{(i)}$ is a **label**.

- Task is to learn a function f mapping inputs x to labels $f(x)$

Generative models

- Learn a distribution $\boldsymbol{p}(\boldsymbol{x}, \boldsymbol{y})$ from training examples
- Often we have $\boldsymbol{p}(\boldsymbol{x}, \boldsymbol{y}) = p(y) * p(x|y)$
- **Output** from the model:

$$\boldsymbol{p}(\boldsymbol{y}, \boldsymbol{x})$$

$$f(x) = \arg\max_y p(y|x) = \arg\max_y \frac{p(y)p(x|y)}{p(x)} \propto \arg\max_y p(y)p(x|y)$$

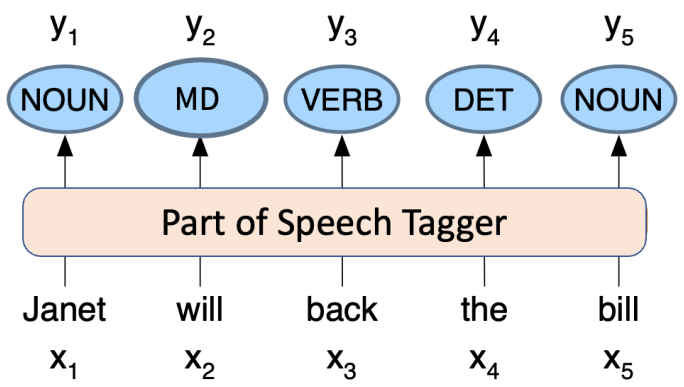
POS Tagging with Hidden Markov Models

- **Input sentence** $x = x_1, x_2, \dots, x_n$

  (x_i is the ith word in the sentence)

- **Tag sequence** $y = y_1, y_2, \dots, y_n$

  (y_i is the ith tag in the sentence)



- We will use HMM to define

$$p(x_1, x_2, \dots, x_n, y_1, y_2, \dots, y_n)$$

for any sentence $x_1, x_2, \dots, x_n$ and tag sequence $y_1, y_2, \dots, y_n$ of the *same length*.

- Then the **most likely tag sequence** for x is

$$\arg\max_{y_1, y_2, \dots, y_n} p(x_1, x_2, \dots, x_n, y_1, y_2, \dots, y_n)$$

Learning of HMM

HMM – A Definition

A **Hidden Markov Model (HMM)** consists of:

- A set $Q = \{q_0, q_1, \ldots, q_T\}$ of **states**, with q_0 the start state.
  (Our non-start states will correspond to parts-of-speech).

- A **transition probability** matrix $A = (a_{ij} \mid 0 \le i \le T, 1 \le j \le T)$,
  where a_{ij} is the probability of jumping from q_i to q_j. For each i, we require $\sum_{j=1}^{T} a_{ij} = 1$.

- For each non-start state q_i and word type w, an **emission probability** $b_i(w)$ of outputting w upon entry into q_i.

We also suppose we're given an **observed sequence** $w_1, w_2 \ldots, w_n$ of word tokens generated by the HMM.

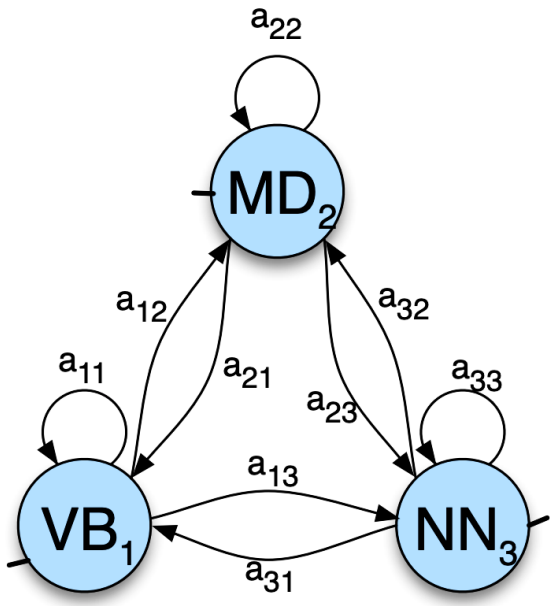
HMM – A Definition

It is easy to generate strings if we know the parameters of the model.

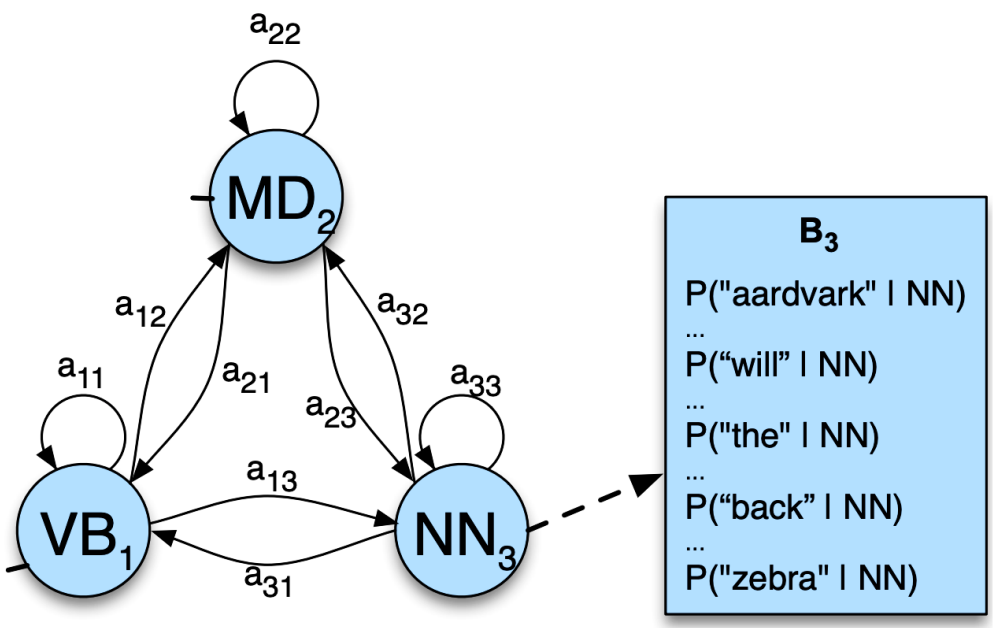
At each time step, make two random choices:

- Use the **transition probabilities** from the previous hidden node to pick the current hidden node.
- Use the **output (emission) probabilities** from the current hidden node to pick the current symbol to output.

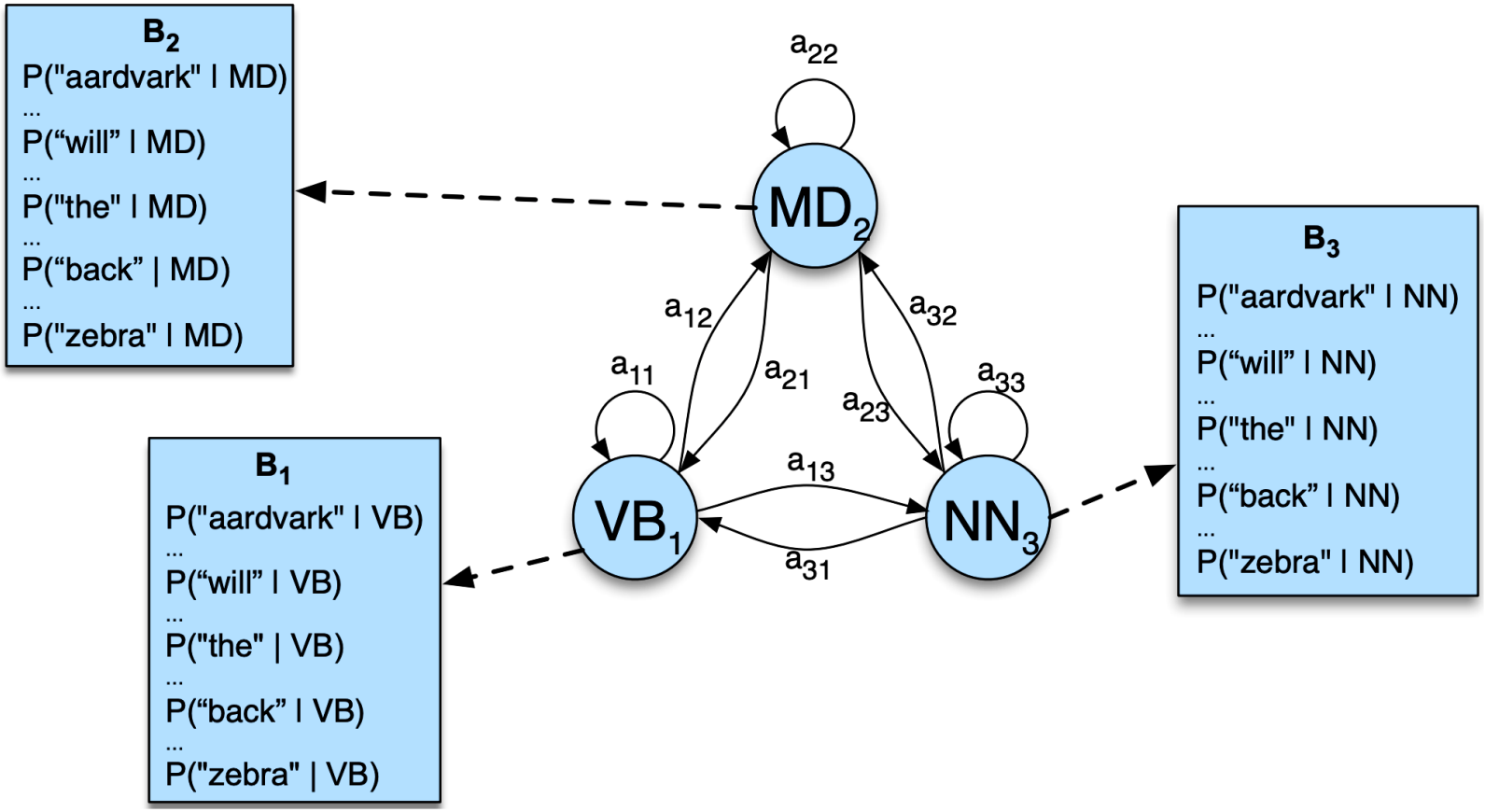
Transition Probabilities



Output (Emission) Probabilities



Output (Emission) Probabilities



HMM – Device to generate sequences

- Hidden Markov models can be thought of as **devices that generate sequences** with **hidden states**:

$$p(\mathbf{NNP} \mid \langle s \rangle)$$

HMM – Device to generate sequences

- Hidden Markov models can be thought of as **devices that generate sequences** with **hidden states**:

NNP

$p(\mathbf{NNP} \mid \langle s \rangle)$

HMM – Device to generate sequences

- Hidden Markov models can be thought of as **devices that generate sequences** with **hidden states**:

NNP

$$p(\textbf{NNP} \mid \langle s \rangle) \times p(\textit{Warwick} \mid \textbf{NNP})$$

HMM – Device to generate sequences

- Hidden Markov models can be thought of as **devices that generate sequences** with **hidden states**:

Warwick
NNP

$$p(NNP \mid \langle s \rangle) \times p(Warwick \mid NNP)$$

HMM – Device to generate sequences

- Hidden Markov models can be thought of as **devices that generate sequences** with **hidden states**:

Warwick	has
NNP	VBZ

$$p(NNP \mid \langle s \rangle) \times p(Warwick \mid NNP)$$

$$p(VBZ \mid NNP) \times p(has \mid VBZ)$$

HMM – Device to generate sequences

- Hidden Markov models can be thought of as **devices that generate sequences** with **hidden states**:

Warwick	has	a
NNP	VBZ	DT

$$p(NNP \mid \langle s \rangle) \times p(Warwick \mid NNP)$$

$$p(VBZ \mid NNP) \times p(has \mid VBZ)$$

$$p(DT \mid VBZ) \times p(a \mid DT)$$

HMM – Device to generate sequences

- Hidden Markov models can be thought of as **devices that generate sequences** with **hidden states**:

Warwick	has	a	very
NNP	VBZ	DT	RB

$$p(NNP \mid \langle s \rangle) \times p(Warwick \mid NNP)$$

$$p(VBZ \mid NNP) \times p(has \mid VBZ)$$

$$p(DT \mid VBZ) \times p(a \mid DT)$$

$$p(RB \mid DT) \times p(very \mid RB)$$

HMM – Device to generate sequences

- Hidden Markov models can be thought of as **devices that generate sequences** with **hidden states**:

Warwick	has	a	very	rich
NNP	VBZ	DT	RB	JJ

$$p(NNP \mid \langle s \rangle) \times p(Warwick \mid NNP)$$
$$p(VBZ \mid NNP) \times p(has \mid VBZ)$$
$$p(DT \mid VBZ) \times p(a \mid DT)$$
$$p(RB \mid DT) \times p(very \mid RB)$$
$$p(JJ \mid RB) \times p(rich \mid JJ)$$

HMM – Device to generate sequences

- Hidden Markov models can be thought of as **devices that generate sequences** with **hidden states**:

Warwick	has	a	very	rich	history
NNP	VBZ	DT	RB	JJ	NN

$$p(NNP \mid \langle s \rangle) \times p(Warwick \mid NNP)$$
$$p(VBZ \mid NNP) \times p(has \mid VBZ)$$
$$p(DT \mid VBZ) \times p(a \mid DT)$$
$$p(RB \mid DT) \times p(very \mid RB)$$
$$p(JJ \mid RB) \times p(rich \mid JJ)$$
$$p(NN \mid JJ) \times p(history \mid NN)$$

How Do we Search for Best Tag Sequence?

- We have defined an HMM, but how do we use it?

- We are **given a word sequence** and must find their corresponding **tag sequence**.

- It's easy to compute the probability of **generating a word sequence $w_1 \ldots w_n$** via a specific **tag sequence $t_1 \ldots t_n$**:

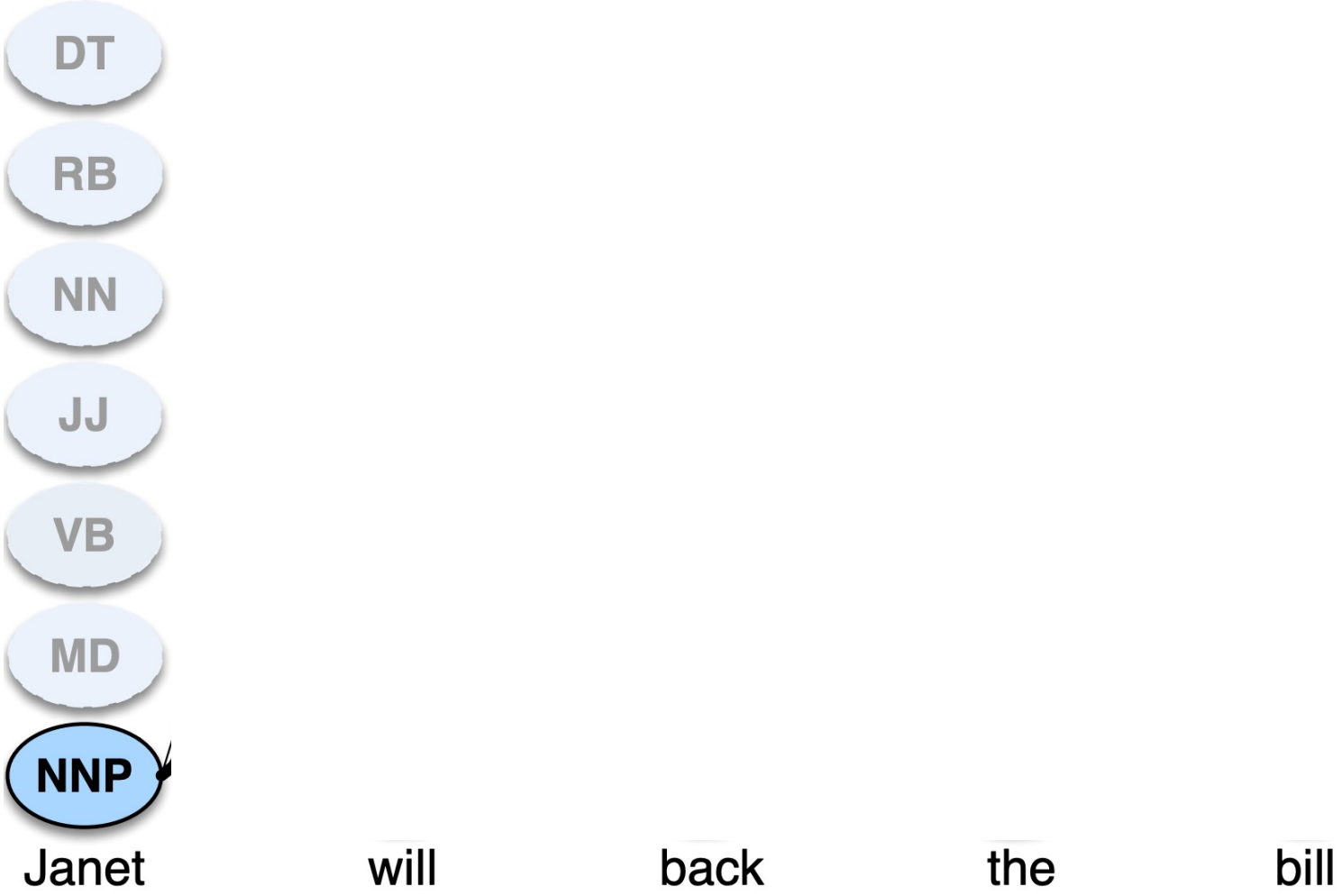
$$\underset{t_1 \ldots t_n}{\operatorname{argmax}} \prod_{i=1}^{n} \overbrace{P(w_i|t_i)}^{\text{emission}} \overbrace{P(t_i|t_{i-1})}^{\text{transition}}$$

- But how do we find the most likely tag sequence?

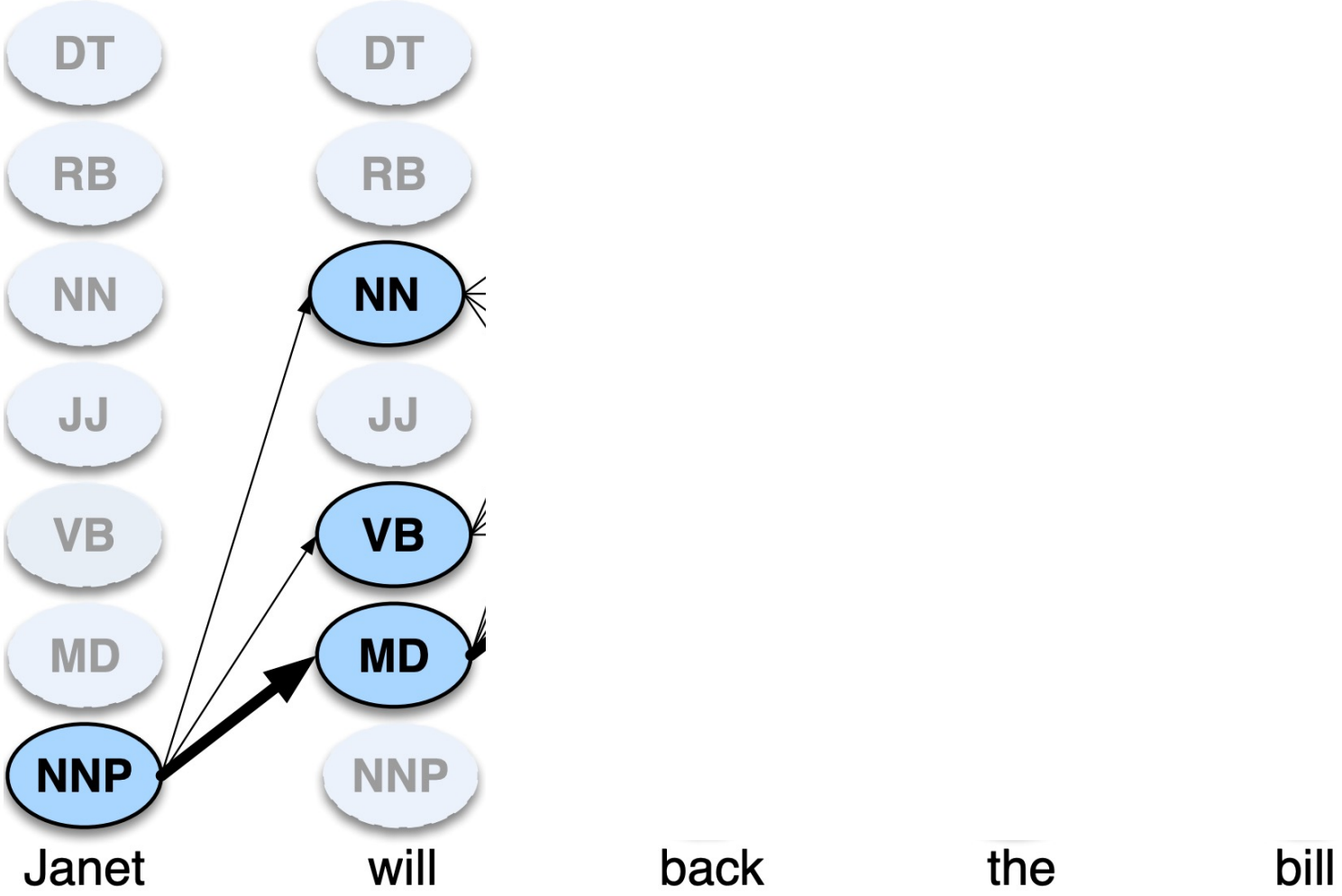
How do we search for best tag sequence?

Janet will back the bill

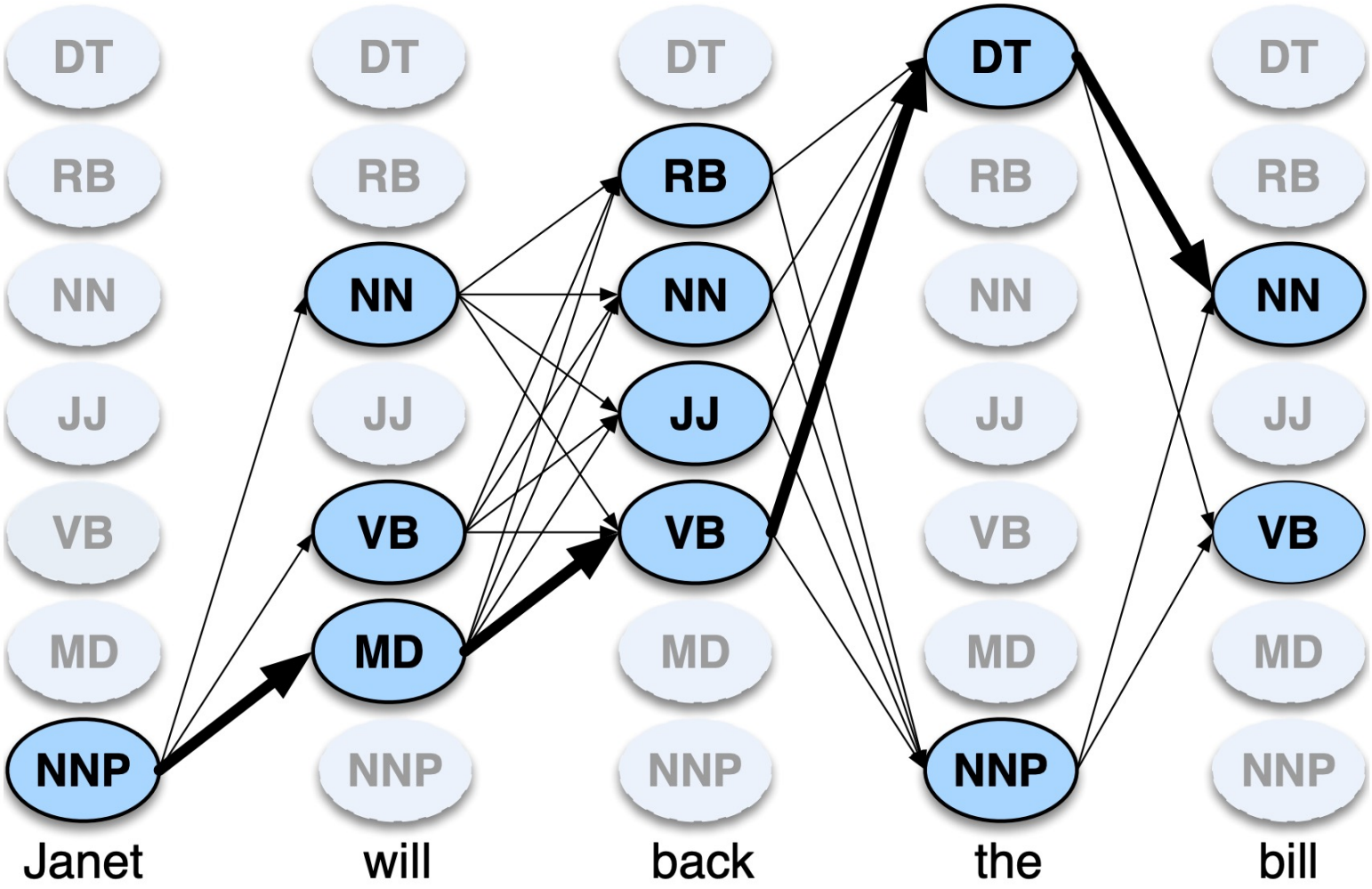
How do we search for best tag sequence?



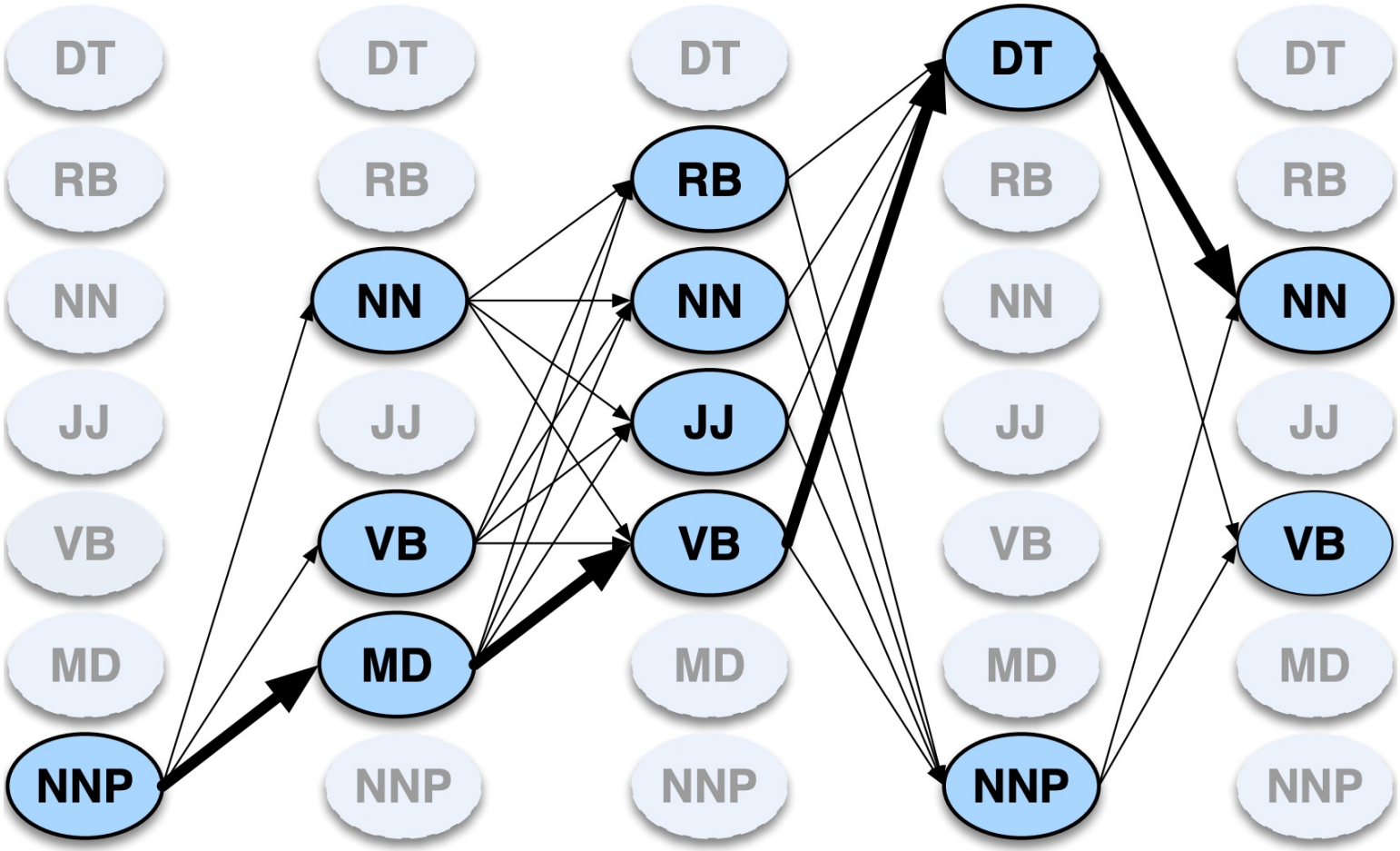
How do we search for best tag sequence?



How do we search for best tag sequence?



How do we search for best tag sequence?



How to predict the **path with higher probability**
when making decisions at the beginning of the sequence?

Decoding

- The task of determining the hidden variables sequence corresponding to the sequence of observations is called **decoding**.

- If have a complete model that can give us what we need, we could just **enumerate all paths** given the input and use the model to assign probabilities to each.
  - Not a good idea.
  - Luckily dynamic programming helps us here

Viterbi Algorithm

- The **Viterbi algorithm** is used to compute **the most likely tag sequence** in $O(T \times N^2)$ time, where N is the number of possible part-of-speech tags and T is the number of words in the sentence.

- The algorithm sweeps through **all the tag possibilities** for each word, computing the best sequence leading to each possibility.

- The key that makes this algorithm efficient is that we only need to know the best sequences leading to the previous word because of the Markov assumption.

Computing the Probability of a Sentence and Tags

- We want to find the sequence of tags that maximizes the formula $P\ (y_1 \ldots y_T | x_1 \ldots x_T)$, which can be estimated as:

$$\prod_{t=1}^{T} P(y_t | y_{t-1}) \times P(x_t | y_t)$$

  - $P(y_t | y_{t-1})$ is computed by multiplying the arc values in the HMM
  - $P(x_t | y_t)$ is computed by multiplying the lexical generation probabilities associated with each word

Computing the Probability of a Sentence and Tags

- Transition Table

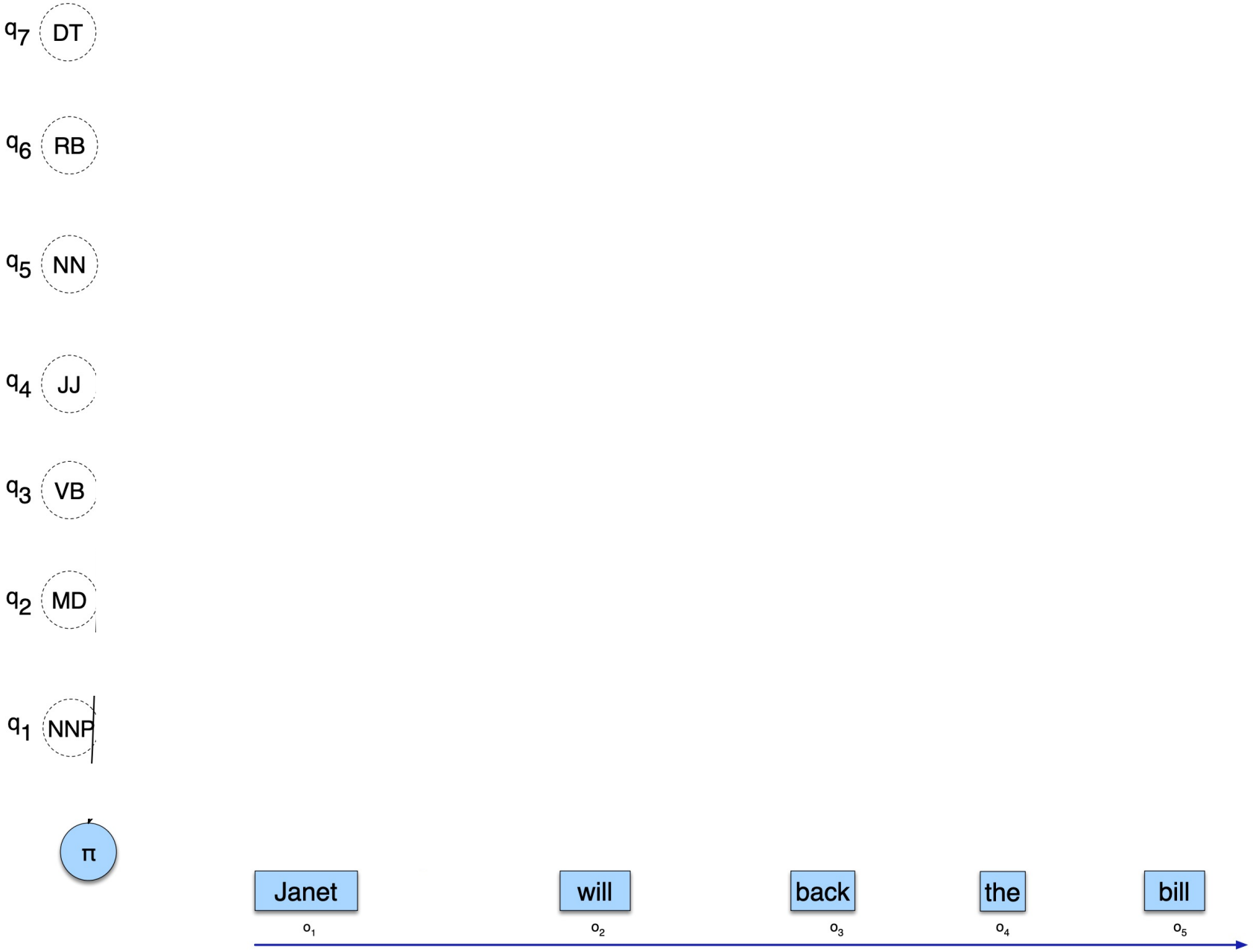
	NNP	MD	VB	JJ	NN	RB	DT
<s>	0.2767	0.0006	0.0031	0.0453	0.0449	0.0510	0.2026
NNP	0.3777	0.0110	0.0009	0.0084	0.0584	0.0090	0.0025
MD	0.0008	0.0002	0.7968	0.0005	0.0008	0.1698	0.0041
VB	0.0322	0.0005	0.0050	0.0837	0.0615	0.0514	0.2231
JJ	0.0366	0.0004	0.0001	0.0733	0.4509	0.0036	0.0036
NN	0.0096	0.0176	0.0014	0.0086	0.1216	0.0177	0.0068
RB	0.0068	0.0102	0.1011	0.1012	0.0120	0.0728	0.0479
DT	0.1147	0.0021	0.0002	0.2157	0.4744	0.0102	0.0017

- Output Table

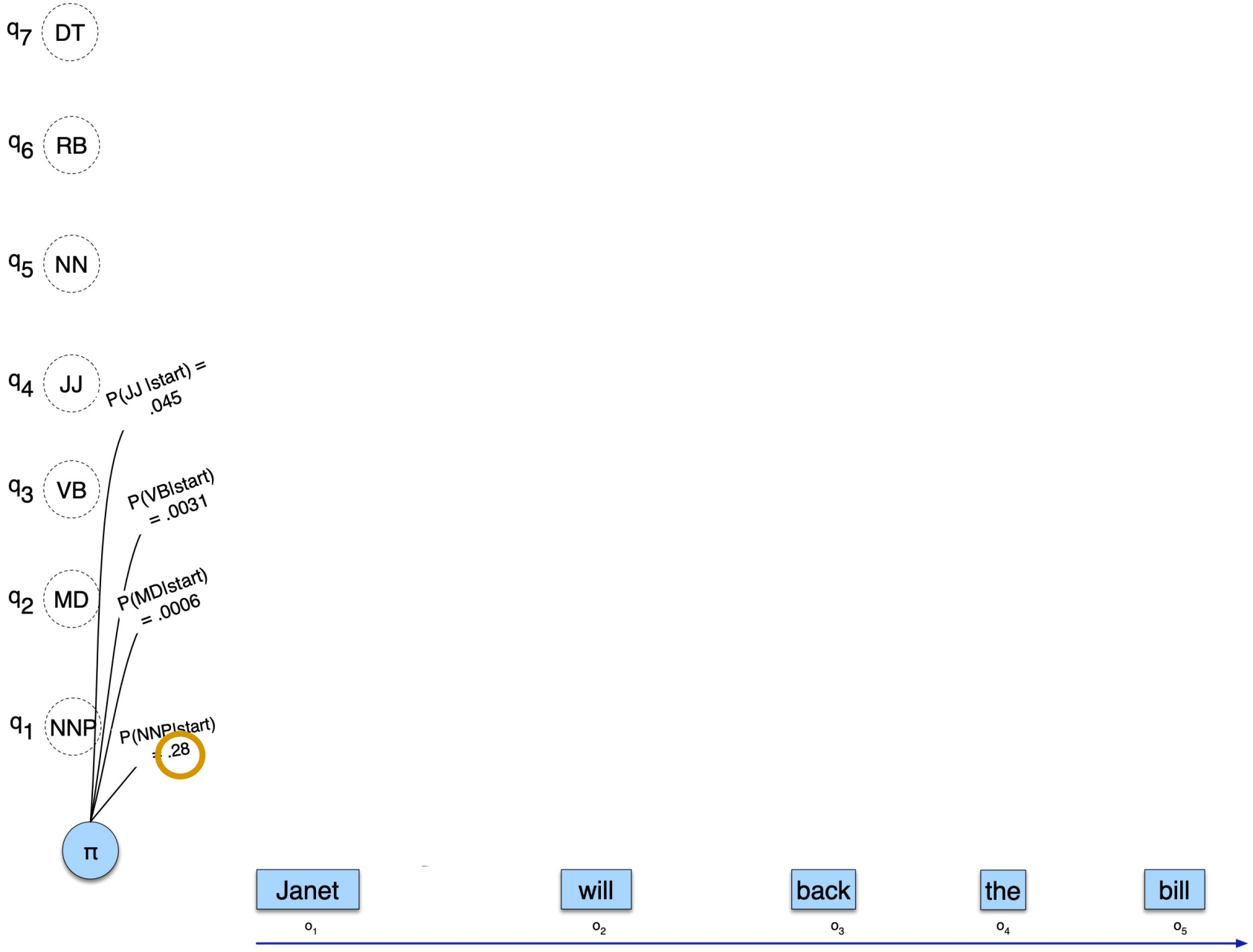
	Janet	will	back	the	bill
NNP	0.000032	0	0	0.000048	0
MD	0	0.308431	0	0	0
VB	0	0.000028	0.000672	0	0.000028
JJ	0	0	0.000340	0	0
NN	0	0.000200	0.000223	0	0.002337
RB	0	0	0.010446	0	0
DT	0	0	0	0.506099	0

- Computed on The Wall Street Journal Dataset

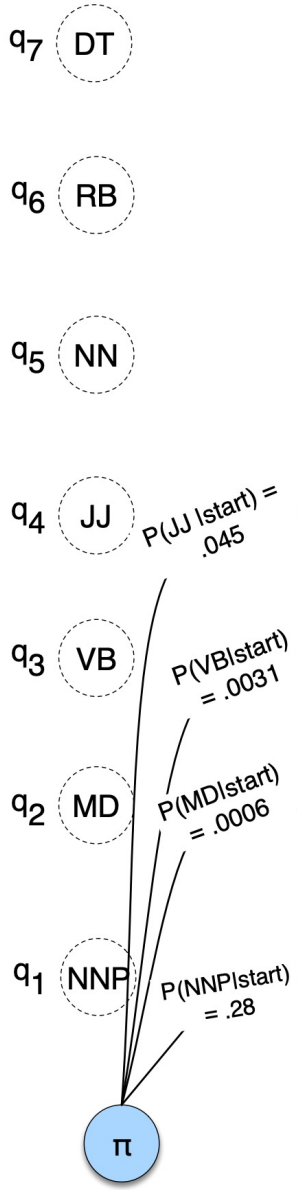
The Viterbi Algorithm



The Viterbi Algorithm



The Viterbi Algorithm

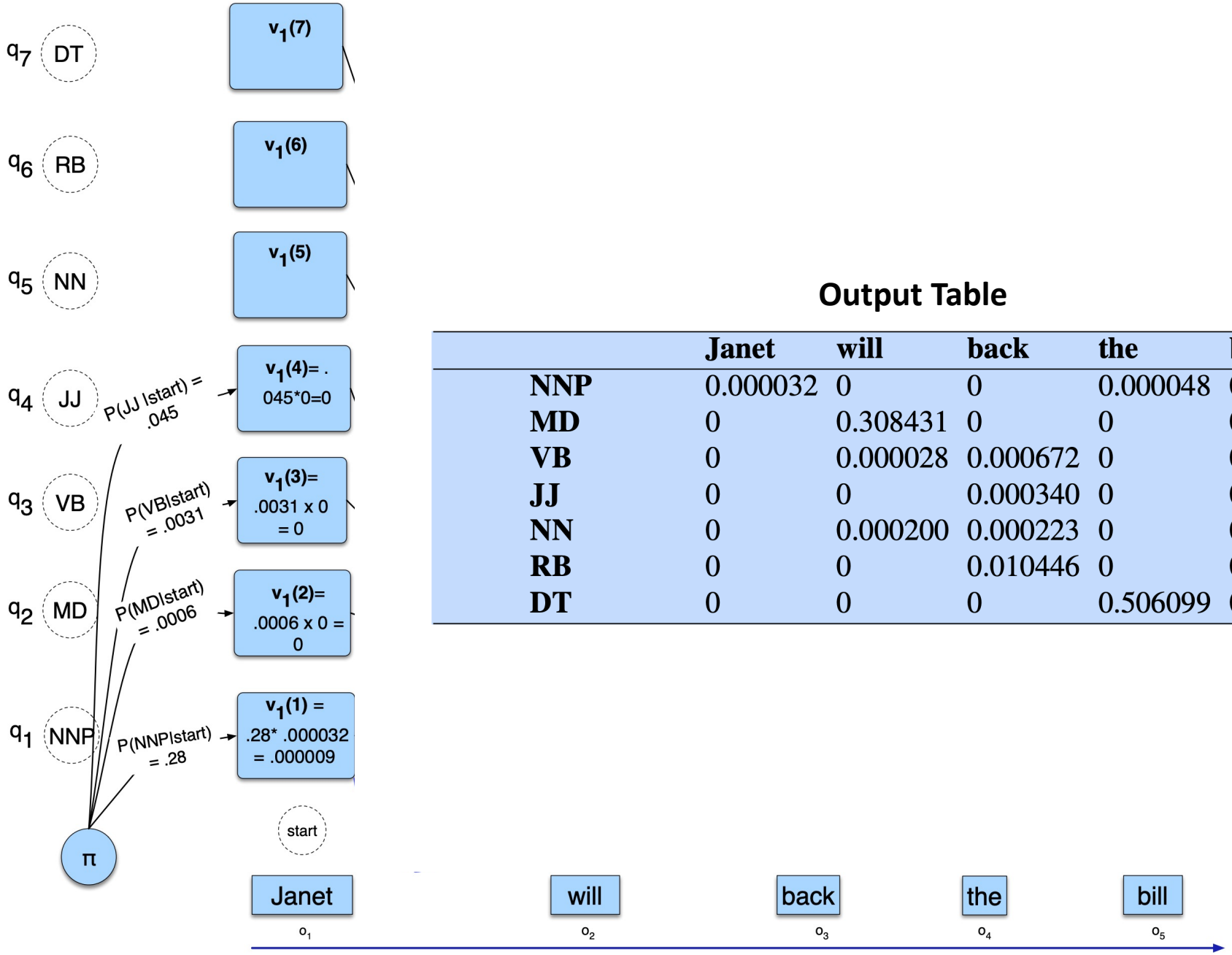


Transition Table

	NNP	MD	VB	JJ	NN	RB	DT
$<s>$	0.2767	0.0006	0.0031	0.0453	0.0449	0.0510	0.2026
NNP	0.3777	0.0110	0.0009	0.0084	0.0584	0.0090	0.0025
MD	0.0008	0.0002	0.7968	0.0005	0.0008	0.1698	0.0041
VB	0.0322	0.0005	0.0050	0.0837	0.0615	0.0514	0.2231
JJ	0.0366	0.0004	0.0001	0.0733	0.4509	0.0036	0.0036
NN	0.0096	0.0176	0.0014	0.0086	0.1216	0.0177	0.0068
RB	0.0068	0.0102	0.1011	0.1012	0.0120	0.0728	0.0479
DT	0.1147	0.0021	0.0002	0.2157	0.4744	0.0102	0.0017

The Viterbi Algorithm

$$\underset{t_1 \ldots t_n}{\operatorname{argmax}} \prod_{i=1}^{n} \overbrace{P(w_i|t_i)}^{\text{emission}} \overbrace{P(t_i|t_{i-1})}^{\text{transition}}$$

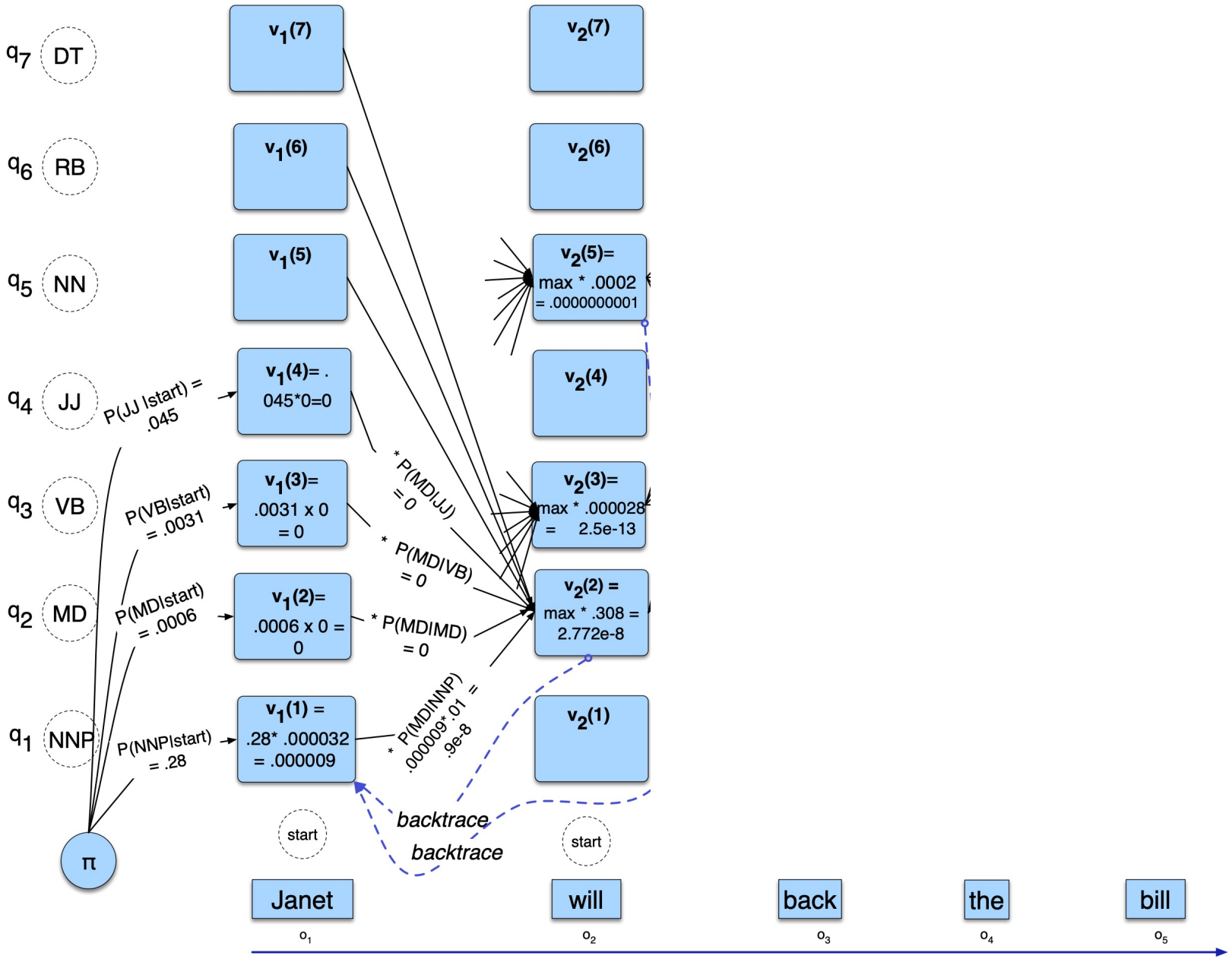


Output Table

	Janet	will	back	the	bill
NNP	0.000032	0	0	0.000048	0
MD	0	0.308431	0	0	0
VB	0	0.000028	0.000672	0	0.000028
JJ	0	0	0.000340	0	0
NN	0	0.000200	0.000223	0	0.002337
RB	0	0	0.010446	0	0
DT	0	0	0	0.506099	0

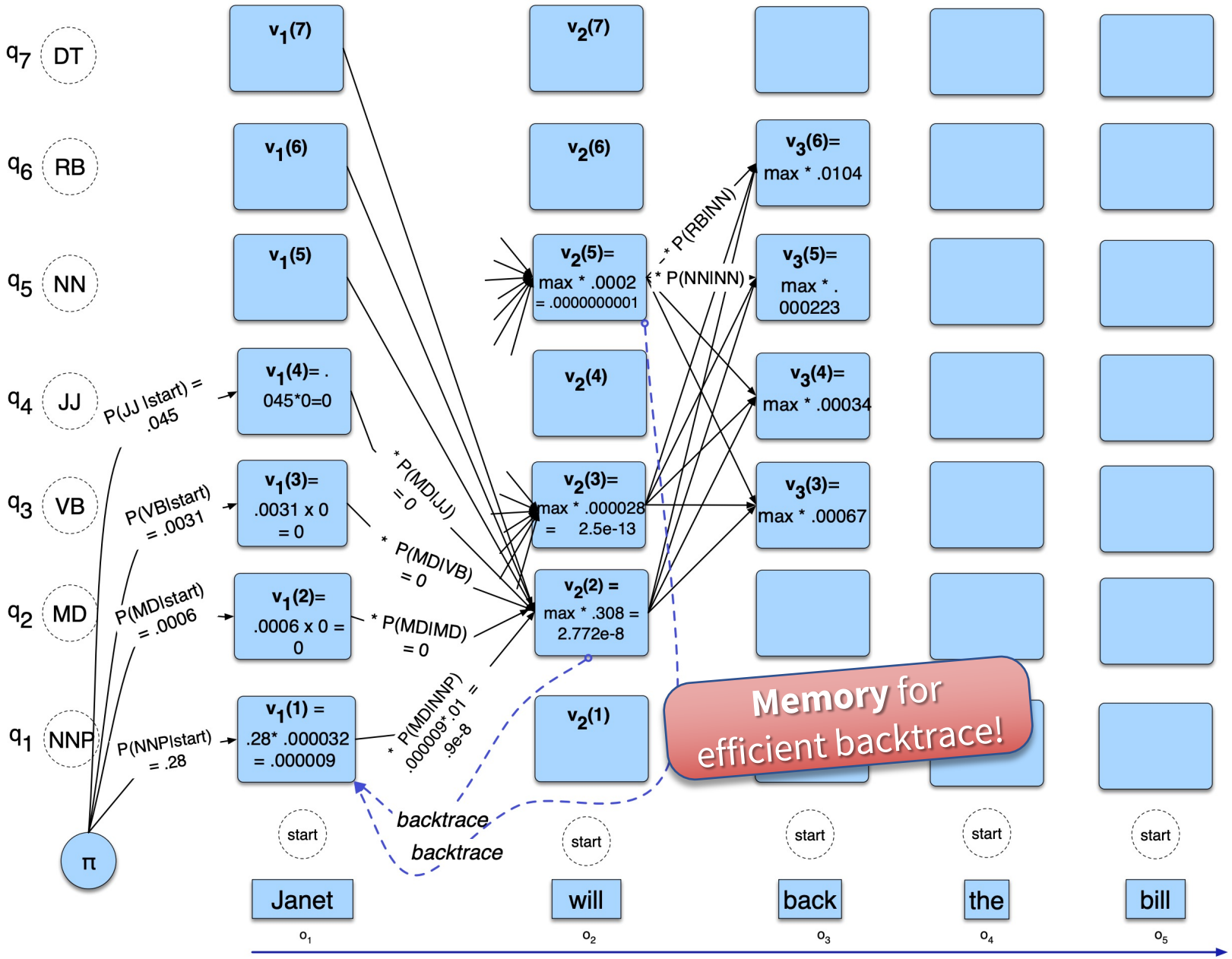
The Viterbi Algorithm

$$\underset{t_1 \ldots t_n}{\operatorname{argmax}} \prod_{i=1}^{n} \overbrace{P(w_i|t_i)}^{\text{emission}} \overbrace{P(t_i|t_{i-1})}^{\text{transition}}$$



The Viterbi Algorithm

$$\underset{t_1 \ldots t_n}{\operatorname{argmax}} \prod_{i=1}^{n} \overbrace{P(w_i|t_i)}^{\text{emission}} \overbrace{P(t_i|t_{i-1})}^{\text{transition}}$$



The Viterbi Algorithm

Let N = # of part-of-speech tags, T = # of words in the sentence

// Iteration Step

for $t = 2$ to T

  for $n = 1$ to N

    $\text{Score}(n,t) = \Pr(Word_t | Tag_n) \times \max_{j=1\ldots N}(\text{Score}(j,t-1) \times \Pr(Tag_n | Tag_j))$

The Viterbi Algorithm

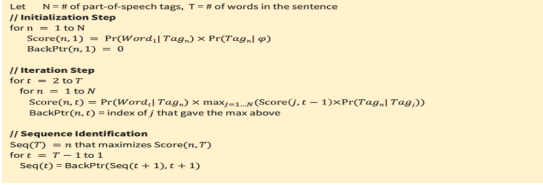
Let N = # of part-of-speech tags, T = # of words in the sentence

// Iteration Step

for $t = 2$ to T

  for $n = 1$ to N

    $\text{Score}(n,t) = \Pr(Word_t | Tag_n) \times \max_{j=1\ldots N}(\text{Score}(j,t-1) \times \Pr(Tag_n | Tag_j))$


Let N = # of part-of-speech tags, T = # of words in the sentence
// Initialization Step
for n = 1 to N
Score$(n,1) = \Pr(Word_1 | Tag_n) \times \Pr(Tag_n | \varphi)$
BackPtr$(n,1) = 0$

// Iteration Step
for $t = 2$ to T
for $n = 1$ to N
Score$(n,t) = \Pr(Word_t | Tag_n) \times \max_{j=1\ldots N}(\text{Score}(j,t-1) \times \Pr(Tag_n | Tag_j))$
BackPtr(n,t) = index of j that gave the max above

// Sequence Identification
Seq$(T) = n$ that maximizes Score(n,T)
for $t = T-1$ to 1
Seq(t) = BackPtr(Seq$(t+1), t+1$)

The Viterbi Algorithm

Let N = # of part-of-speech tags, T = # of words in the sentence

// Initialization Step

for n = 1 to N

$$\text{Score}(n, 1) = \Pr(Word_1 | Tag_n) \times \Pr(Tag_n | \varphi)$$

// Iteration Step

for t = 2 to T

for n = 1 to N

$$\text{Score}(n, t) = \Pr(Word_t | Tag_n) \times \max_{j=1\ldots N}(\text{Score}(j, t-1) \times \Pr(Tag_n | Tag_j))$$

Let N = # of part-of-speech tags, T = # of words in the sentence

// Initialization Step

for n = 1 to N

$\text{Score}(n, 1) = \Pr(Word_1 | Tag_n) \times \Pr(Tag_n | \varphi)$

$\text{BackPtr}(n, 1) = 0$

// Iteration Step

for t = 2 to T

for n = 1 to N

$\text{Score}(n, t) = \Pr(Word_t | Tag_n) \times \max_{j=1\ldots N}(\text{Score}(j, t-1) \times \Pr(Tag_n | Tag_j))$

$\text{BackPtr}(n, t)$ = index of j that gave the max above

// Sequence Identification

$\text{Seq}(T) = n$ that maximizes $\text{Score}(n, T)$

for $t = T - 1$ to 1

$\text{Seq}(t) = \text{BackPtr}(\text{Seq}(t+1), t+1)$

The Viterbi Algorithm

Let N = # of part-of-speech tags, T = # of words in the sentence

// Initialization Step

for n $=$ 1 to N

$\text{Score}(n, 1) = \Pr(Word_1 | Tag_n) \times \Pr(Tag_n | \varphi)$

// Iteration Step

for $t = 2$ to T

for $n = 1$ to N

$\text{Score}(n, t) = \Pr(Word_t | Tag_n) \times \max_{j=1\ldots N}(\text{Score}(j, t-1) \times \Pr(Tag_n | Tag_j))$

// Sequence Identification

$\text{Seq}(T) = n$ that maximizes $\text{Score}(n, T)$

for $t = T - 1$ to 1

$\text{Seq}(t) = \text{BackPtr}(\text{Seq}(t+1), t+1)$

Let N = # of part-of-speech tags, T = # of words in the sentence
// Initialization Step
for n = 1 to N
$\text{Score}(n, 1) = \Pr(Word_1 | Tag_n) \times \Pr(Tag_n | \varphi)$
$\text{BackPtr}(n, 1) = 0$
// Iteration Step
for $t = 2$ to T
for $n = 1$ to N
$\text{Score}(n, t) = \Pr(Word_t | Tag_n) \times \max_{j=1\ldots N}(\text{Score}(j, t-1) \times \Pr(Tag_n | Tag_j))$
$\text{BackPtr}(n, t)$ = index of j that gave the max above
// Sequence Identification
$\text{Seq}(T) = n$ that maximizes $\text{Score}(n, T)$
for $t = T - 1$ to 1
$\text{Seq}(t) = \text{BackPtr}(\text{Seq}(t+1), t+1)$

The Viterbi Algorithm

Let N = # of part-of-speech tags, T = # of words in the sentence

// Initialization Step

for n $=$ 1 to N

$\text{Score}(n, 1) = \Pr(Word_1 | Tag_n) \times \Pr(Tag_n | \varphi)$

$\text{BackPtr}(n, 1) = 0$

// Iteration Step

for $t = 2$ to T

for $n = 1$ to N

$\text{Score}(n, t) = \Pr(Word_t | Tag_n) \times \max_{j=1\ldots N}(\text{Score}(j, t-1) \times \Pr(Tag_n | Tag_j))$

BackPtr(n, t) = index of j that gave the max above

// Sequence Identification

Seq$(T) = n$ that maximizes Score(n, T)

for $t = T - 1$ to 1

Seq(t) = BackPtr(Seq$(t+1), t+1$)

Let N = # of part-of-speech tags, T = # of words in the sentence
// Initialization Step
for n = 1 to N
$\text{Score}(n, 1) = \Pr(Word_1 | Tag_n) \times \Pr(Tag_n | \varphi)$
$\text{BackPtr}(n, 1) = 0$
// Iteration Step
for $t = 2$ to T
for $n = 1$ to N
$\text{Score}(n, t) = \Pr(Word_t | Tag_n) \times \max_{j=1\ldots N}(\text{Score}(j, t-1) \times \Pr(Tag_n | Tag_j))$
BackPtr(n, t) = index of j that gave the max above
// Sequence Identification
Seq$(T) = n$ that maximizes Score(n, T)
for $t = T - 1$ to 1
Seq(t) = BackPtr(Seq$(t+1), t+1$)

Evaluation

Once you have you POS tagger running how do you evaluate it?

- Overall error rate with respect to a gold-standard **test set**.
- Error rates on **particular tags**
- Error rates on **particular words**
- Tag confusions...

Error Analysis

- Look at a confusion matrix

	IN	JJ	NN	NNP	RB	VBD	VBN
IN	—	.2			.7		
JJ	.2	—	**3.3**	2.1	1.7	.2	**2.7**
NN		**8.7**	—				.2
NNP	.2	**3.3**	**4.1**	—	.2		
RB	**2.2**	2.0	.5		—		
VBD		.3	.5			—	**4.4**
VBN		**2.8**				**2.6**	—

- See what errors are causing problems
  - Noun (NN) vs Adj (JJ) / Proper Noun (NNP) vs Noun (NN) / NNP vs. JJ
  - Verb past tense (VBD) vs Verb past participle (VBN) / VBN vs JJ

Evaluation

- Current POS tagging system are very accurate:
  - Baseline system that predicts the most common POS for a word already gets 90% accuracy → thanks to many words not being ambiguous.
- Standard classifiers (no sequence) can achieve 93% accuracy
- Sequence classifiers can achieve 97% accuracy

Reading

- Jurafsky, Daniel and James H. Martin. 2026. Speech and Language Processing. 3rd edition. **Chapters 17, A.**

- Bird Steven, Ewan Klein, and Edward Loper. Natural Language Processing with Python. O'Reilly Media, Inc., 2009. **Chapter 6.**

- List of software for POS tagging: https://aclweb.org/aclwiki/Part-of-speech_tagging